CSC311: Introduction to Machine Learning

Project Report

Submitted by

Victor Jiang, Yuelin Wu, William Penn

University of Toronto
Winter 2026

1 Data Exploration

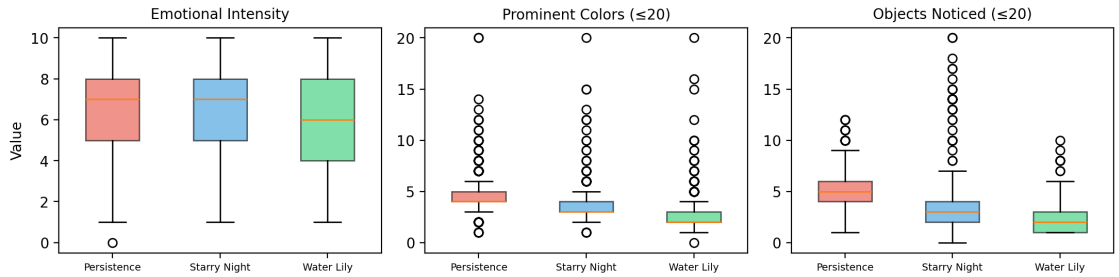

Figure 1: Boxplots of various numeric features by target painting, after removal of single outlier that answered 100 for both "prominent amount of colours" and "objects noticed"

First, we interpreted "On a scale of 1-10, how intense is the emotion conveyed by the artwork?" as a numeric feature, as the intensity is in a range of 1-10, and number being close indicates a similar level of intensity. Looking at *Figure 1*, the distribution of emotional intensity was relatively similar across all three paintings, with the mean for "Water Lily" of around 6 being lower than the mean emotional intensity for "Persistence" and "Starry Night", both around 7. Our interpretation of this is that a lower emotional intensity may be slightly more correlated with "Water Lily" than the other two paintings.

Both "How many prominent colours do you notice in this painting?" and "How many objects caught your eye in the painting?" are counts that are best represented as numeric features. The boxplots show that "Persistence" has a slightly higher mean for both features than "Starry Night", which also has a slightly higher mean than "Water Lily". This suggests that a higher count of prominent colours and objects noticed may be slightly more likely correlated with "Persistence" and "Starry Night" than "Water Lily."

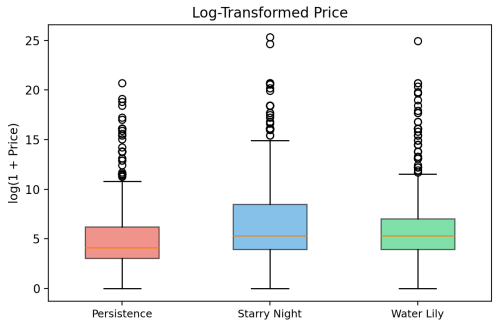

Figure 2: Boxplots of log-transformed price by target painting, after removal of two outliers that answer 10^{60} and 10^{80}

We interpreted "How much (in Canadian dollars) would you be willing to pay for this painting?" as a numeric feature as well, as the price is a continuous value. However, we log-transformed the price by $\log_{10}$ as many prices are incredibly high, making it difficult to meaningfully interpret. From *Figure 2*, we can see that the distribution is relatively similar across all three paintings. Thus, we determined that the log-price may not be a very useful feature, but we included it regardless.

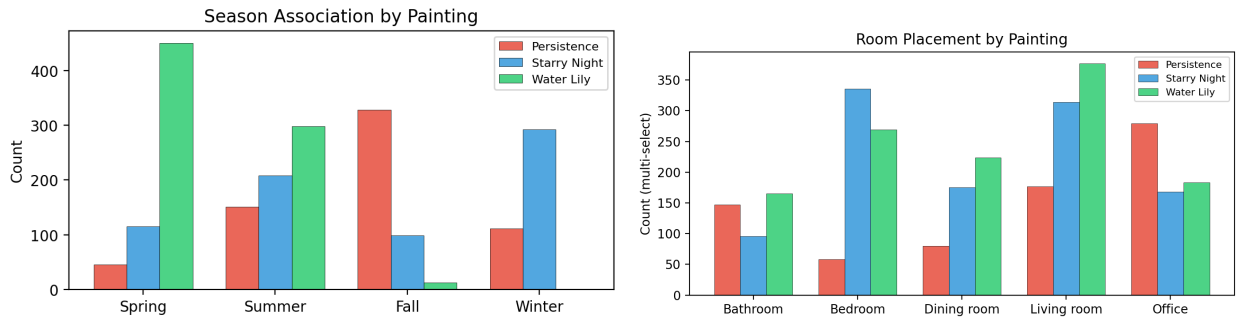

Figure 3: Histogram of "Season" and "Room Placement" by target painting

We decided that "What season does this art piece remind you of?" should be treated as a categorical feature that are encoded as one-hot vectors, as all answers are one of the four seasons. From *Figure 3*, we can clearly see that Spring is highly correlated with "Water Lily", Fall is highly correlated with "Persistence", and Winter is highly correlated with "Starry Night"; Summer is more so correlated with "Water Lily" than the others, but not as strongly. From this, we expect that this feature will be very useful for our model. We decided that "If you could purchase this painting, which room would you put that painting in?" is also obviously a categorical feature that should be encoded as one-hot vectors. Although it is more difficult to interpret the histogram, there are clearly correlations between various room placements and the three paintings. The "If you could view this art in person, who would you want to view it with?" feature is extremely similar and was encoded as one-hot vectors as well.

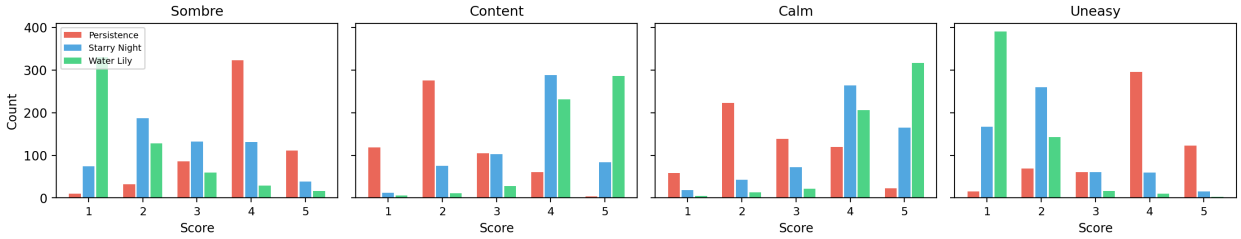

Figure 4: Histogram of the four likert features by target painting

We interpreted the four likert questions as essentially numeric features, as the answers are on a scale of 1-5 and the number being close indicates a similar level of feeling. There are some clear correlations present, such high "content" and "calm" feelings being more correlated with "Water Lily".

Lastly, we decided to use a Bag of Words representation implemented with `sklearn`'s `CountVectorizer` for the three free response questions (description, food, soundtrack) as they are all open-ended questions with a variety of answers. We ended up not including the soundtrack feature as through ablation where we excluded the soundtrack feature from our model, we found that removing the soundtrack feature actually improved our model performance, suggesting that the soundtrack feature may be noisy and not very useful for our model.

We chose to split our data into 80% training set and 20% validation set, stratified so that each subset maintains the same proportion of samples from each class of target paintings. Considering the limited amount of data we are working with and the fact that a lot of the data is incomplete, instead of using a separate test set and diminishing the size of our training/validation sets, we decided to use cross-validation to evaluate our model performance. [1]

2 Model Exploration

We looked at a variety of models that seemed most likely to give the best results: logistic regression, random forest classifiers, naive bayes, and MLPs. For logistic regression, we standardized the numeric features as we are performing regularization, and then we tuned the (inverse) regularization strength hyperparameter C. For random forest classifiers, the data input was fine the way it was, and we tuned both the number of estimators (trees in the forest) as well as the maximum depth of the trees.

We also explored using a naive bayes model. Since our data is both numeric and categorical, we explored using a Gaussian naive bayes model, which also required standardization of the numeric features. As well, we explored using a Complement naive bayes model, which was recommended by `sklearn` for imbalanced datasets and for text classification, which could be helpful for parsing the BoW features [2]. Lastly, we explored using a MLP model, which also required standardization of the numeric features. We tuned both the hidden layer sizes and the regularization strength α.

3 Model Choice and Hyperparameters

To give each model family the best chance of performing well, we evaluated our models by looking at the combination of hyperparameters that gave the model within each model family the highest validation accuracy. For example, our hyperparameter tuning for the random forest model with 200 trees can be seen below in *Figure 5*, and our hyperparameter tuning for the logistic regression model can be seen below in *Figure 6*. We then compared the best models from each family by evaluating their stratified 5-fold cross-validation accuracy, which acted effectively as our test set and remained consistent across all models.

We found that the models with the best cross-validation accuracy were a MLP with one hidden layer of 64 and $\alpha = 0.01$ that had an accuracy of 0.8766 ± 0.0111, a random forest classifier with 200 trees and a depth of 10 that had an accuracy of 0.8772 ± 0.0116, and finally a logistic regression model with $C = 0.1$ that had an accuracy of 0.8796 ± 0.0090. As the logistic regression model had the highest cross-validation accuracy and was also the simplest model to implement, we chose the logistic regression model as our final model.

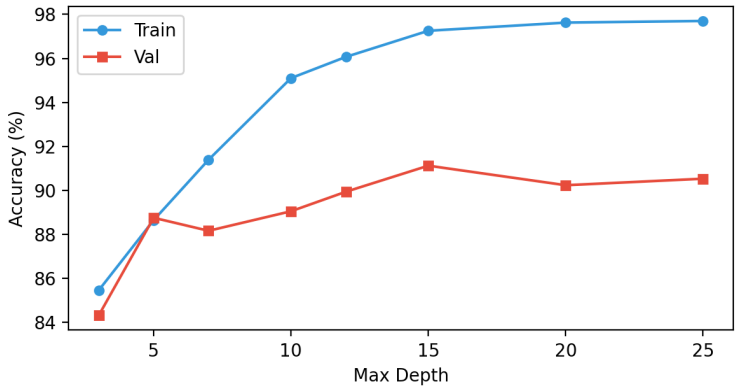

Figure 5: Accuracy of random forest with 200 trees for various depths

For the logistic regression model, the only hyperparameter that required tuning was the inverse regularization strength C. We started with a large range of values for C from 0.01 to 10.0 as seen in *Figure 6*, and then narrowed down our search to values between 0.01 and 0.2. We found that either $C = 0.05$ or $C = 0.1$ would give us the best validation accuracy of 0.8935; we ultimately chose $C = 0.1$ as it also had the highest training accuracy, as well as being the highest cross-validation accuracy.

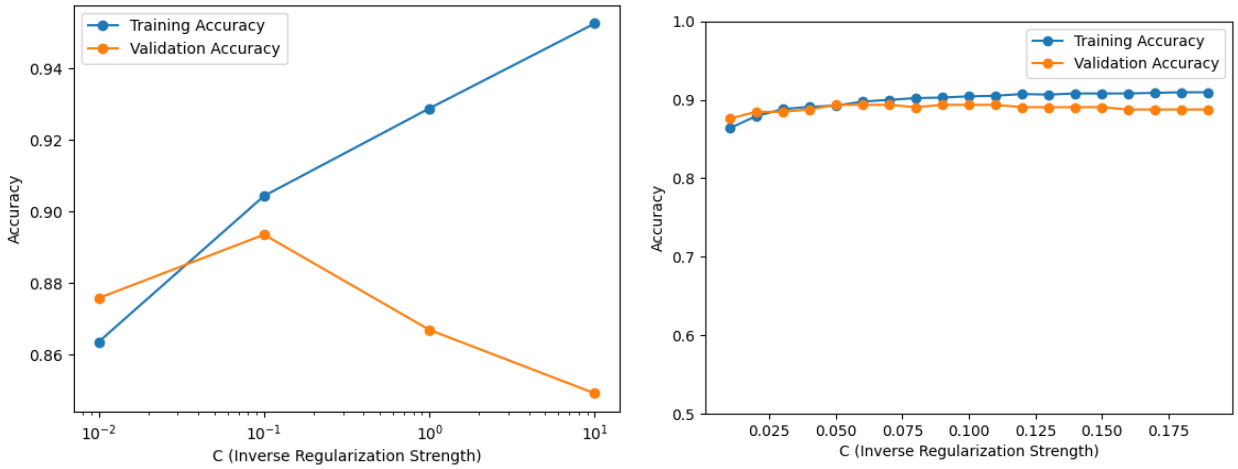

Figure 6: Accuracy of logistic regression for various C

Therefore, our final model choice for this challenge is a Logistic Regression model with an inverse regularization strength of $C = 0.1$. After selecting this model, we considered what other hyperparameters we could tune, as well as any other optimizations we could make to improve model performance. First, we realized that another hyperparameter we could tune was the way we are representing the free response questions as BoW features. We could decide to use a TF-IDF representation instead of a count representation, or we could decide to use bigrams instead of unigrams; we could also tune the number of features we are including in our BoW representation.

After iterating through numerous combinations of these BoW hyperparameters, we found that the best combination was consistently using a count representation and unigrams.

We then iterated through various values for the maximum number of features to include, and found that the best value was around 200 features as seen in *Figure 7*.

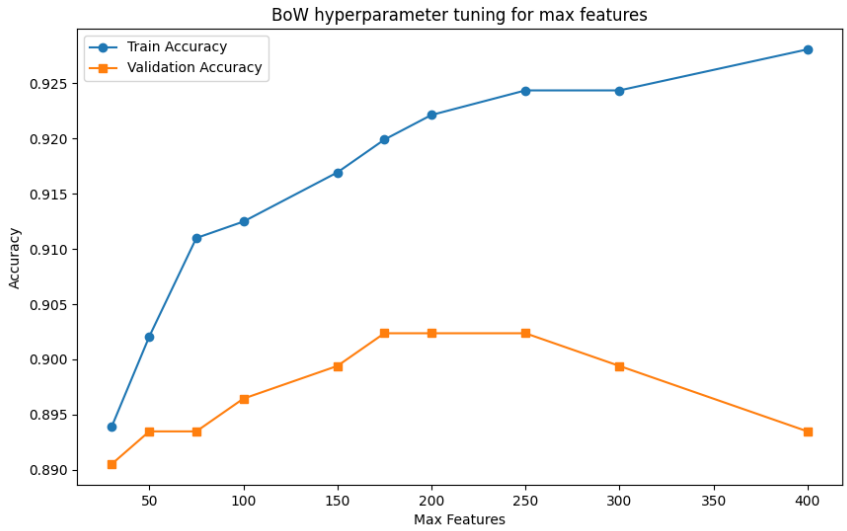

Figure 7: Bag of Words max feature amount hyperparameter tuning for a Count representation with unigrams

Finally, we performed an ablation study to see whether the removal of any features would actually improve our model performance. As seen in *Figure 8*, we found that the removal of the soundtrack BoW feature actually improved our model performance by 0.3%, suggesting that the soundtrack feature may be noisy and not very useful for our model. This matches with empircal evaluation of the dataset, as there are very varied answers to the soundtrack question, and it is difficult to find a clear correlation between the soundtrack answers and the target paintings. Thus, we decided to remove the soundtrack BoW feature from our final model.

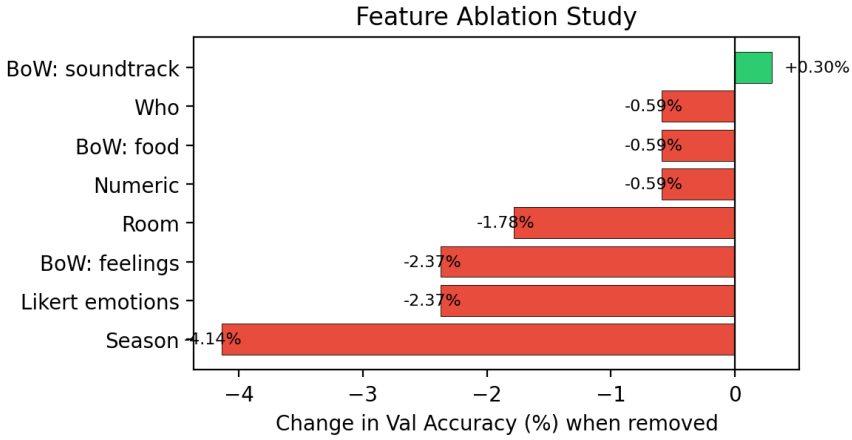

Figure 8: Feature Ablation Study resulting in the removal of the soundtrack BoW feature

4 Prediction

After again training our final logistic regression model on the training set, we evaluated the model using cross-validation—which acted effectively as our test set—and found that our final model had a cross-validation accuracy of 0.8855 ± 0.0070. Since this cross-validation result is a reasonable estimate for real world performce, we estimate that our model will also perform with an accuracy of 0.8855 on the final test set. [1]

5 Workload Distribution

- Victor Jiang:
- Yuelin Wu: I was mainly responsible with the hyperparameter tuning for the Random Forest and Logistic Regression models, as well as the tuning for the BoW features. I also wrote the final report, with input from my teammates.
- William Penn:

References

[1] scikit learn. Cross-validation: evaluating estimator performance. `https://scikit-learn.org/stable/modules/cross_validation.html`. Accessed: 2026-04-05.

[2] scikit learn. Naive bayes. `https://scikit-learn.org/stable/modules/naive_bayes.html`. Accessed: 2026-04-05.